

Controlul traficului in rețele de calculatoare. Planificarea pachetelor

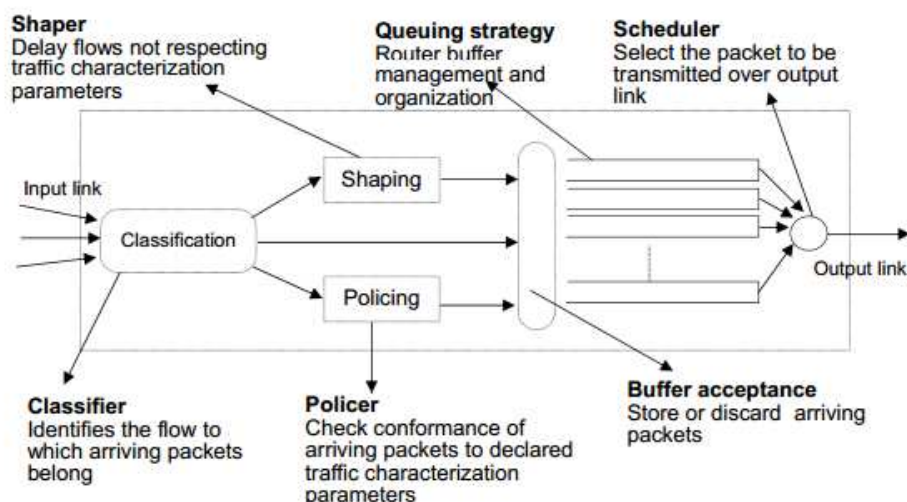
Obiectivul lucrării

Lucrarea își propune introducerea tehnicilor de planificare a pachetelor, compararea a acestora prin exemple teoretice si exerciții. Aplicațiile vor viza implementarea acestor algoritmi in C sub sistemul de operare Linux si observarea comparativa a modului de funcționare.

Breviar teoretic

Planificarea pachetelor este ultima etapa de procesare înaintea transmiterii pachetelor catre interfața de iesire. Planificatorul este un arbitru (bloc de decizie) ce hotaraste care pachet stocat in cozile de asteptare va fi trimis spre ieșire.

QoS-capable router

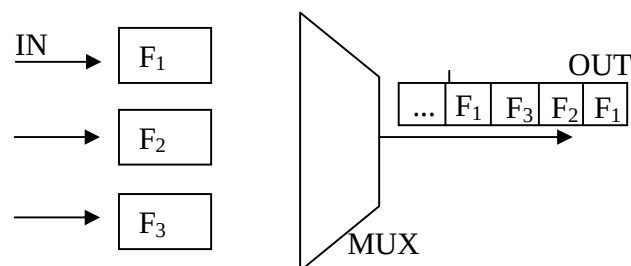


Se urmaresc in luarea deciziilor: asigurarea vitezei; incararea echilibrata (load balancing) intre conexiuni concurente; garantarea ratei de pierderi;garantarea intarzierii si jitterului.

Metode de planificare:

FIFO

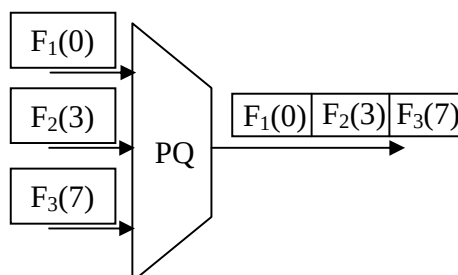
- Routerelor tradiționale au o singura coada de așteptare asociata interfeței de ieșire;



Algoritmul este simplu (costuri reduse), nu ofera protectie impotriva fluxurilor non-standard si nu poate garanta întârzierea si viteza de transfer;

Priority Queuing (PQ)

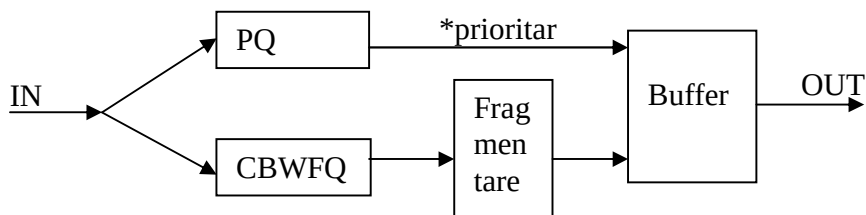
- Aceste routere analizeaza campul prioritate din antetul IP. In IP v6 acesta are valori de la 0-7 in care 0 este asociat traficului neclasificat, iar 7 este asociat traficului de prioritate maxima (exp. Traficul de routere);



Dezavantaje: poate bloca fluxurile cu prioritate redusa; timpul mediu de intarziere este dependent de valoarea MTU (maximum transmission unit).

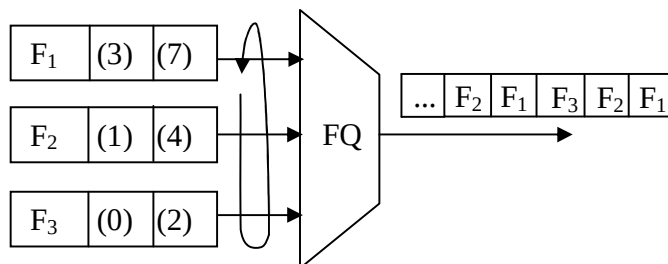
4. Load Balancing Queuing

Acest mecanism combina metoda de clasificare dupa prioritate si metode de prioritizare in functie de clasa. Traficul asociat cozilor tratate cu algoritmul PQ este servit prioritar inaintea celui CBWFQ.



5. FQ (Fair Queuing)

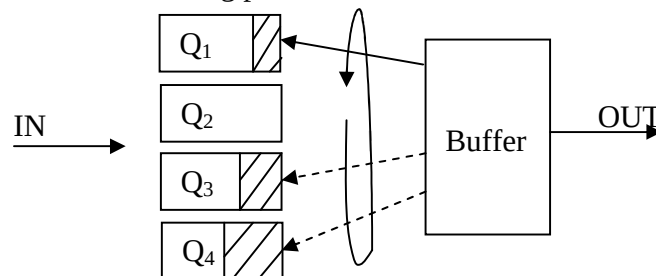
- Pentru aceasta tehnica / metoda planificatorul isi adaptează dinamic disciplina de servire a cozilor;
- In fiecare etapa de transmisie orice flux are dreptul sa transmită un pachet;
- Coada de asteptare va fi reorganizata pentru fiecare flux in parte;



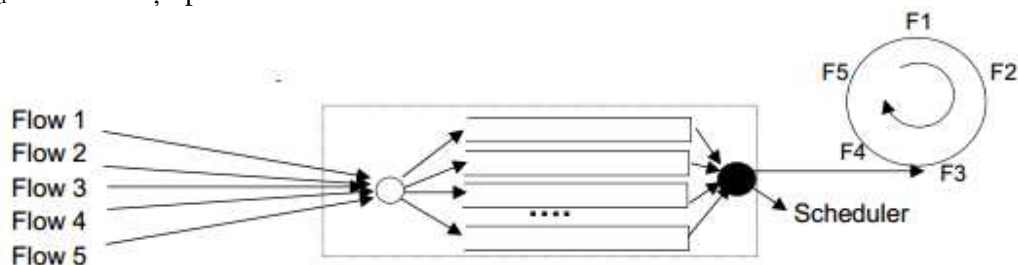
- FQ asigura un tratament egal pentru toate cozile de asteptare;
- Daca dimensiunea pachetului este fixa (retele ATM) se asigura o alocare echitabila a vitezei de transmisie;
- Daca pachetele au dimensiuni diferite sunt favorizate pachetele de dimensiuni mari.

1. Round Robin

Este un algoritm care incearca sa fie echitabil cu toate fluxurile de intrare. Foloseste un buffer organizat in cozi de asteptare separate, o coada pentru fiecare flux (fiecare coada este organizata FIFO). Algoritmul permite unui router sa organizeze mai multe cozi de asteptare. Pachetele se extrag pe rand din fiecare coada care nu este goala.



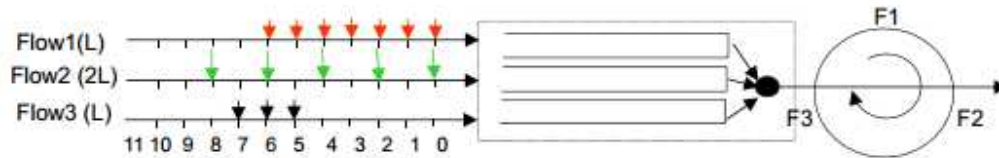
Q_i = cozi de așteptare



Aceasta este metoda de utilizare a bufferilor hardware din placa de retea.

Dimensiunea pachetelor influenteaza complexitatea implementarii. Dezavantajul acestui algoritmul este ca acorda latime de banda mai mare echipamentelor care folosesc pachete de dimensiune mare. O solutie alternativa ce rezolva aceasta problema este prelucrarea datelor octet cu octet.

Round Robin: example



T	In F1	In F2	In F3	Q(F1)	Q(F2)	Q(F3)	Scheduled packets
0	P10[1]	P20[2]	-	P10	P20	-	F1:P10
1	P11[1]	-	-	P11	P20	-	F2:P20
2	P12[1]	P22[2]	-	P11,P12	P22	-	F2:P22 (cont)
3	P13[1]	-	-	P11,P12,P13	P22	-	F1:P11
4	P14[1]	P24[2]	-	P12,P13,P14	P22,P24	-	F2:P22
5	P15[1]	-	P35[1]	P12,P13,P14,P15	P24	P35	F2:P22 (cont)
6	P16[1]	P26[2]	P36[1]	P12,P13,P14,P15,P16	P24,P26	P35,P36	F3:P35
7	-	-	P37[1]	P12,P13,P14,P15,P16	P24,P26	P36,P37	F1:P12
8	-	P28[2]	-	P13,P14,P15,P16	P24,P26	P36,P37	F2:P24
9	-	-	-	P13,P14,P15,P16	P26,P28	P36,P37	F2:P24
10	-	P2A[2]	-	P13,P14,P15,P16	P26,P28	P36,P37	F3:P36

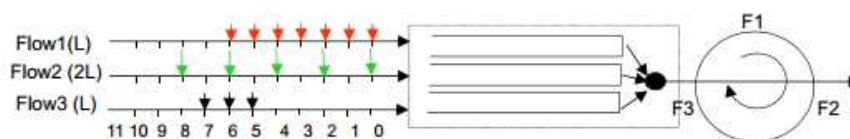
Deficit Round Robin

Acest algoritm foloseste o variabila pentru a determina numărul de biti dintr-o coada de asteptare care trebuie preluate într-o rundă. Dacă fiecare coadă de asteptare are pachete de transmis, iar lungimea pachetului este mai mică sau egală cu valoarea variabilei, atunci pachetul este transmis și se decrementează variabila cu lungimea pachetului.

Dacă lungimea pachetului este mai mare decât valoarea variabilei se așteaptă runda următoare în care variabila este incrementată în mod automat.

Algoritmul poate defavoriza unele cozi de transmisie într-o rundă însă acestea vor recupera în una din rundele următoare.

Deficit Round Robin: example



T	Inc.	D[1]	D[2]	D[3]	Q(F1)	Q(F2)	Q(F3)	Scheduled
0	F1	0+1-1	0	0	P10	P20	-	F1:P10
1	F2,F1	0+1-1	0+1	0	P11	P20	-	F1:P11
2	F2	0	1+1-2	0	P12	P22	-	F2:P20
3	-	0	0	0	P12,P13	P22	-	F2:P20(cont)
4	F1	0+1-1	0	0	P13,P14	P22,P24	-	F1:P12
5	F2,F3	0	0+1	0+1-1	P13,P14,P15	P22,P24	P35	F3:P35
6	F1	0+1-1	1	0	P14,P15,P16	P22,P24,P26	P36	F1:P13
7	F2	0	1+1-2	0	P14,P15,P16	P22,P24,P26	P36,P37	F2:P22
8	-	0	0	0	P14,P15,P16	P24,P26	P36,P37	F2:P22(cont)
9	F3	0	0	0+1-1	P14,P15,P16	P24,P26	P36,P37	F3:P36
10	F1	0+1-1	0	0	P15,P16	P24,P26	P37	F1:P14
11	F2,F3	0	0+1	0+1-1	P15,P16	P24,P26	P37	F3:P37
...								

Aplicație

Planificatorul Deficit Round Robin (DRR) neponderat

Informatii mentinute de planificatorul Deficit Round Robin (DRR) neponderat:

- **Deficit counter** = the **number of bytes** that a **flow** is **allowed to transmit** when it is its turn.
- **Quantum** = the **number of bytes** that is **added** to the **deficit counter** of flow in **each round** (**Quantum** = **amount of credit per round**)

Modul de funcționare al planificatorului DRR neponderat:

- Packets from **flows** are **transmitted** in a **round robin** manner
- The **quantum** is **added** to the **deficit counter** of a flow **before servicing a flow**.

- $\text{deficit counter}(f) = \text{deficit counter}(f) + \text{quantum}$
- A **packet** from a **flow** is **only transmitted** if:

- $\text{deficit counter} \geq \text{length of packet}$
- If a **packet** is **transmitted**, the **deficit counter** of that **flow** is **updated** as follows:

- $\text{deficit counter}(f) = \text{deficit counter}(f) - (\text{\#bytes in packet})$

Nota:

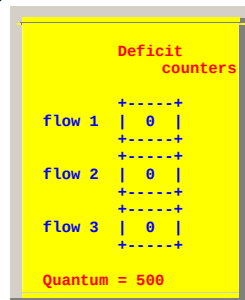
- When the **DDR router** detects the **first packet** of a **new flow**, it **allocates a deficit counter** for the **new flow** and **initialize the deficit counter to zero**

0 Exemple: planificatorul DRR neponderat:

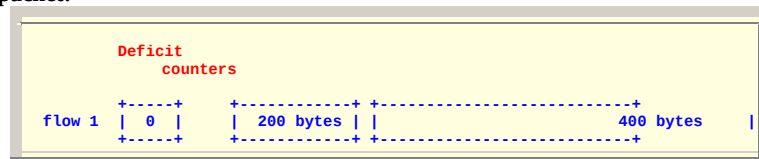
0 Exemplu:

- **3 flows**
- **Quantum = 500 (bytes)**

Grafic:



0 Sosire pachet:



flow 2	0	100	200	300 bytes
flow 3	0	200 bytes	200	200

Quantum = 500

0 Deservire flux 1:

- Add quantum to flow 1's **deficiy counter**:

Deficit counter				
flow 1	500	200 bytes	400 bytes	
flow 2	0	100	200	300 bytes
flow 3	0	200 bytes	200	200

Quantum = 500

- Transmit **first packet** from Flow 1:

Deficit counter				
flow 1	300	400 bytes		
flow 2	0	100	200	300 bytes
flow 3	0	200 bytes	200	200

Quantum = 500

Fluxul 1 nu are suficiente credite pentru pachetul urmator si isi pierde randul.
Fluxul 1 va transporta "transmisia" reziduala pentru urmatoarea runda de servicii.

0 Deservire flux 2:

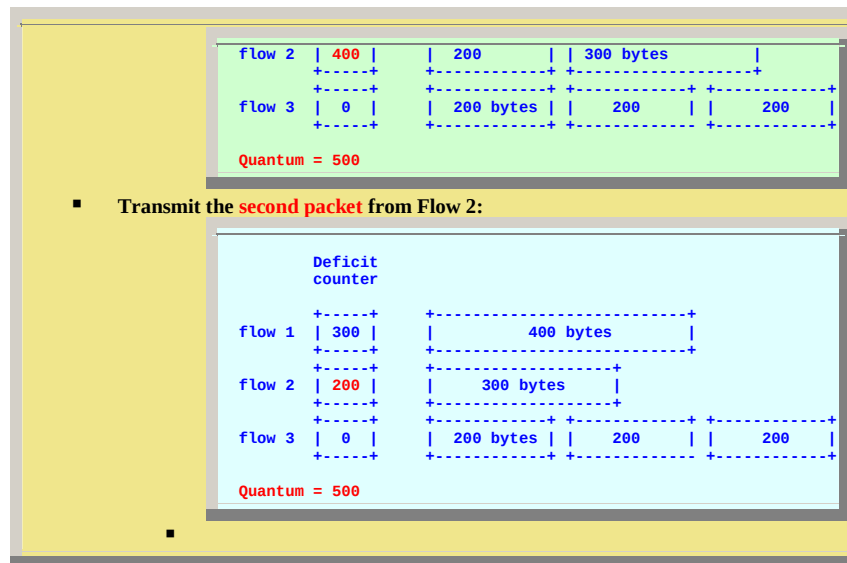
- Add quantum to flow 2's **deficiy counter**:

Deficit counter				
flow 1	300	400 bytes		
flow 2	500	100	200	300 bytes
flow 3	0	200 bytes	200	200

Quantum = 500

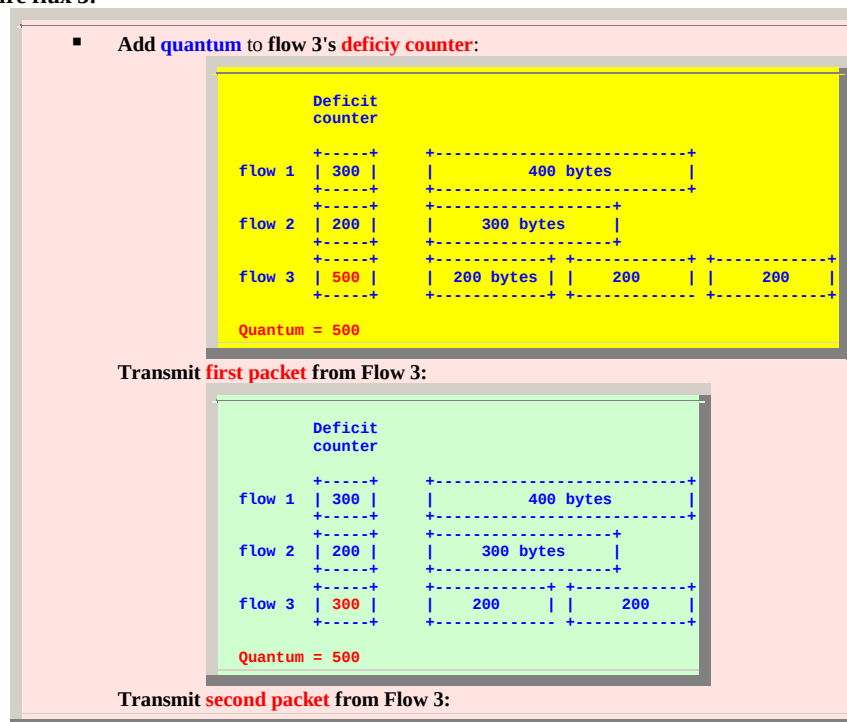
- Transmit **first packet** from Flow 2:

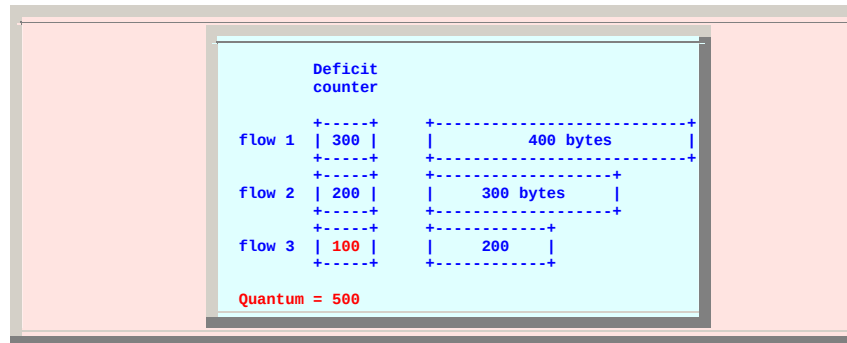
Deficit counter				
flow 1	300	400 bytes		



Fluxul 1 nu are suficiente credite pentru pachetul urmator si isi pierde randul.
 Fluxul 1 va transporta “transmisia” reziduala pentru urmatoarea runda de servicii.

Deservire flux 3:

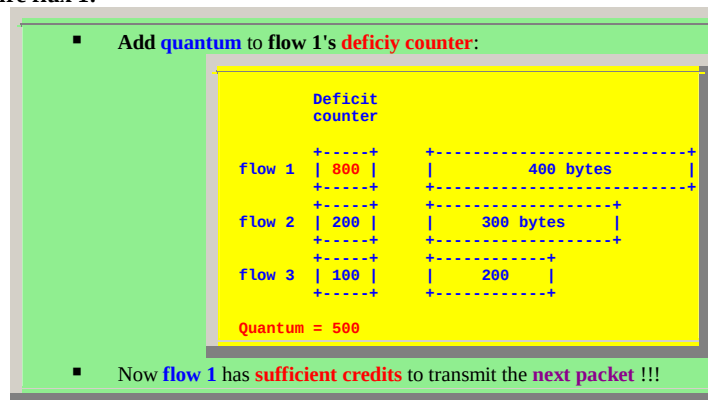




- Fluxul 3 nu are suficiente credite pentru pachetul urmator si isi pierde randul.
- Fluxul 3 va transporta “transmisia” reziduala pentru urmatoarea runda de servicii.

0

0 Deservire flux 1:



0 Si asa mai departe . . .

0 Din calcul, ar trebui sa fie clar ca:

- Daca este nesaturat fluxul va impartii latimea de banda in mod corect
- Fluxurile saturate (fluxurile ce transmit la o viteza mai mica decat ponderea corecta) vor receptiona viteza de serviciu completa .
- Restul vitezei de transmisie va fi impartit la fluxurile nesaturate.

Desfășurarea lucrării

1. Se va studia breviarul teoretic.
2. Se vor implementa pentru fiecare dintre algoritmi prezentați cate o aplicație in C sub Linux.
3. Se vor observa avantajele si dezavantaje fiecarui algoritm.